Universidad Tecnológica Nacional – Facultad Regional Tucumán

Tecnicatura Universitaria en Desarrollo y Producción de Videojuegos

Cátedra: Programación de Videojuegos I
Profesor: Matías Chocobar
Trabajo Práctico Grupal – Semana 2: De la Consola al Motor Gráfico (Unity 2D y Físicas)
FECHA LÍMITE DE ENTREGA: VIERNES 30 DE OCTUBRE DE 2026 HASTA LAS 23:50 HS. (Vía Pull Request en GitHub)

---

1. ¿De qué se trata esta entrega?

¡Buenas a todos! Llegamos a la Semana 2 y marcamos un hito importante en la materia. En la primera semana trabajaron en la arquitectura pura en C#: diseñaron clases sólidas para manejar la vida, los puntos de daño, las estadísticas del personaje y los inventarios. Todo eso funcionaba tras bambalinas, imprimiendo textos por consola.

Ahora da inicio la etapa de integración gráfica: nos trasladamos a **Unity 2D** para dar vida visual e interactiva a ese código.

Aclaramos un punto fundamental para que encaren esto con tranquilidad: **no estamos buscando un juego terminado**, con menú principal, arte final pulido o música de estudio. Lo que evaluamos en esta entrega es un prototipo técnico jugable (Greybox/Whitebox). Queremos ver que su personaje se mueva de forma fluida, responda a las leyes físicas del motor, colisione correctamente con el entorno e interactúe con los objetos de la escena utilizando las clases C# que ya construyeron.

La dinámica de trabajo: Rotación de Proyectos

En la industria profesional de videojuegos es extremadamente común entrar a un proyecto ya iniciado o recibir código escrito por otros programadores. Para simular

esta experiencia real, en esta entrega aplicamos **rotación de repositorios**: a cada grupo le tocará continuar el desarrollo sobre el proyecto del grupo anterior.

- **Si el código que reciben está ordenado y bien documentado:** ¡Excelente! Tienen un gran punto de partida para construir la fase 2.
- **Si el código recibido tiene fallas, está incompleto o desorganizado:** No entren en pánico. Su primer trabajo será refactorizarlo, corregir los errores necesarios y dejar constancia de estas mejoras en el informe técnico.
- **Aclaración clave:** Nadie va a perder puntos por las fallas, fallos de lógica o código incompleto del grupo anterior. Lo que evaluaremos es cómo ustedes diagnostican el estado inicial, resuelven los problemas y construyen la nueva entrega sobre esa base.

---

2. Reglas de oro de la Cátedra (¡Para no perder puntos!)

Para asegurar que sus proyectos mantengan un estándar profesional de arquitectura y físicas, deben respetar las siguientes cinco reglas técnicas de manera estricta:

- **Regla 1: Prohibido meter toda la lógica en un solo script gigante (No a las God Classes).**
  Las clases de C# puro que crearon en la Semana 1 (`Jugador`, `Inventario`, `Enemigo`, etc.) siguen vivas. No las conviertan a `MonoBehaviour` si no es necesario. El script de Unity debe actuar únicamente como un **puente** (controlador) que toma el *input* del jugador, mueve los componentes físicos y le consulta o actualiza los datos al modelo lógico puro en C#.
- **Regla 2: Cero teletransportación con `transform.position` en objetos con físicas.**
  Si un GameObject tiene un componente `Rigidbody2D` de tipo *Dynamic*, **jamás** modifiquen su posición directamente mediante `transform.position = ...`. Esto rompe la simulación del motor de físicas, genera traspasos no deseados de paredes (*tunneling*) y elimina la inercia. El movimiento debe realizarse asignando `rigidbody2d.velocity` o aplicando fuerzas con `rigidbody2d.AddForce()`.
- **Regla 3: Teclas en `Update()`, físicas en `FixedUpdate()`.**
  La lectura de teclado o gamepad (`Input.GetAxisRaw`,

`Input.GetButtonDown`, etc.) se debe realizar dentro del método `Update()`, ya que este corre al ritmo de los cuadros por segundo del monitor. Toda la aplicación de movimiento, cambios de velocidad o fuerzas físicas debe ejecutarse exclusivamente dentro de `FixedUpdate()`, que corre a un ritmo de reloj fijo en el motor físico. Si leen teclas dentro de `FixedUpdate()`, Unity se comerá pulsaciones de teclas.

- **Regla 4: Bloquear la rotación en Z.**
  En los ajustes del componente `Rigidbody2D` del personaje principal, recuerden siempre activar la opción **Constraints → Freeze Rotation Z**. De lo contrario, cuando el personaje choque contra un escalón u obstáculo, comenzará a rodar y caerá de cabeza en el escenario.
- **Regla 5: Detección de suelo bien implementada.**
  Para habilitar el salto, no usen booleanos simples que cambien en cualquier contacto. Deben verificar que el contacto sea efectivamente un suelo. Pueden lograrlo evaluando el vector normal del contacto (`contact.normal.y > 0.6f` dentro de `OnCollisionStay2D`) o utilizando técnicas de `Physics2D.Raycast` / `Physics2D.OverlapCircle`. Esto evita que el personaje salte repetidamente en el aire o se quede pegado a las paredes laterales al presionar la tecla de salto.

---

3. Temas que vemos y aplicamos en clase

Durante las sesiones teóricas y prácticas de esta semana abordamos los siguientes pilares de Unity 2D, los cuales deben verse aplicados en su entrega:

1. **GameObjects, Transform y scripts MonoBehaviour:** Composición de objetos en escena, jerarquías de transformaciones y ciclo de vida de un script (`Awake`, `Start`, `Update`, `FixedUpdate`).
2. **Movimiento con Vector2 y normalización:** Manipulación de vectores bidimensionales. Aprendimos a aplicar `.normalized` a los vectores de movimiento para evitar el clásico problema de andar más rápido en diagonal al sumar los ejes X e Y.
3. **Rigidbody2D y sus tipos:**
   - **Dynamic:** Objeto afectado totalmente por la gravedad y las fuerzas físicas (el jugador, cajas empujables).

- **Kinematic:** Objeto con física pero no afectado por gravedad ni fuerzas externas; se mueve por código (plataformas móviles, proyectiles sencillos).
- **Static:** Objeto inamovible de la escena que bloquea a otros (paredes, pisos).

4. **Colisiones sólidas vs. Triggers/sensores:**
   - **Colisiones sólidas (`OnCollisionEnter2D`):** Producen una respuesta física física dura (rebote, freno).
   - **Triggers/Sensores (`OnTriggerEnter2D`):** La casilla `Is Trigger` está activada. Los objetos se atraviesan pero el motor detecta la superposición. Ideal para recolectables, zonas de daño, zonas de carga o detectores.

---

4. Tabla de asignación de temas (Sorteo de rotación)

A continuación se detalla la rotación de temas para cada grupo en esta Semana 2:

Grupo	Tema Asignado (Semana 2)	Tema Heredado (Semana 1)
Grupo 1	Tema 7: Dark Souls 2D	Tema 1
Grupo 2	Tema 4: Street Fighter 2D	Tema 2
Grupo 3	Tema 8: Resident Evil 2D	Tema 3
Grupo 4	Tema 1: Super Mario Bros 2D	Tema 4
Grupo 5	Tema 9: Hollow Knight 2D	Tema 5
Grupo 6	Tema 2: The Legend of Zelda 2D	Tema 6
Grupo 7	Tema 10: Pac-Man 2D	Tema 7
Grupo 8	Tema 5: Doom 2D	Tema 8
Grupo 9	Tema 6: Minecraft 2D	Tema 9

Grupo	Tema Asignado (Semana 2)	Tema Heredado (Semana 1)
Grupo 10	Tema 3: Pokémon 2D	Tema 10
Grupo 11	Tema 8 (Variación Supervivencia / Horda)	Tema 1

---

5. Cómo trabajar con Git y GitHub

Para mantener la prolijidad en el repositorio central de la cátedra, deberán seguir estos pasos estrictos de control de versiones:

1. **Sincronizar el repositorio:** El líder del grupo debe hacer un **Sync Fork** en la interfaz de GitHub o ejecutar `git merge upstream/main` desde su terminal local para traer todos los cambios acumulados de la Semana 1.
2. **Uso obligatorio de ramas personales:** Queda estrictamente prohibido commitear directamente sobre la rama `main`. Cada integrante del equipo debe crear y trabajar sobre su propia rama de desarrollo siguiendo este patrón de nombre: `dev-NombreApellido-Legajo` (por ejemplo: `dev-JuanPerez-54321`).
3. **Integración:** Una vez completada y probada la tarea individual, cada integrante integra sus cambios hacia la rama de desarrollo del grupo.
4. **Entrega final:** El líder de grupo es el encargado de abrir el **Pull Request (PR)** hacia el repositorio principal de la cátedra.
    - **Formato obligatorio del título del PR:** `[ENTREGA FASE 2] Grupo XX - Tema: NombreDelJuego`

---

6. El Reporte Técnico (`reporte.md`)

Dentro de la carpeta del grupo en la raíz del repositorio, deben crear o actualizar el archivo `reporte.md` utilizando formato Markdown. Debe contener los siguientes ítems redactados de forma clara:

- **Integrantes y Legajos:** Nómina completa de los miembros del grupo que trabajaron en esta fase.
- **Diagnóstico del proyecto heredado:** Informe sincero y constructivo de cómo encontraron el proyecto del grupo anterior. ¿El código C# estaba ordenado? ¿Faltaban clases? ¿Había errores de compilación?
- **Refactorizaciones y arreglos:** Lista de correcciones que tuvieron que realizar sobre el código recibido para poder avanzar con la integración de Unity.
- **Integración C# con Unity:** Explicación técnica breve de cómo conectaron las clases puras en C# (Lógica) con los componentes `MonoBehaviour` (Vista/Controlador).
- **Guía de prueba de la escena:** Instrucciones simples para que los docentes probemos la escena en Unity (qué escena abrir, qué teclas presionar y qué comportamientos observar).

---

7. Enunciados de los 10 juegos

A continuación se detallan las consignas específicas para la implementación mecánica de cada juego:

Tema 1: Super Mario Bros 2D

- **Plataformas y física:** Configurar un personaje con `Rigidbody2D` Dynamic. Implementar un salto que conserve la inercia horizontal.
- **Bloques interactivos:** Crear bloques con `Collider2D` estático que, al ser golpeados desde abajo (detectado con una colisión o raycast con normal hacia abajo), ejecuten un método que incremente las monedas en la clase de C# pura.
- **Enemigo (Goomba):** Crear un objeto enemigo. Si el jugador lo pisa desde arriba (normal del contacto `contact.normal.y < -0.5f`), el enemigo se destruye y da impulso hacia arriba al jugador. Si lo toca desde un lateral, invoca el método de recibir daño de la clase `Jugador`.
- **Zona de muerte:** Colocar una zona vacía al fondo del mapa con un `Collider2D` configurado como `Is Trigger` que elimine al jugador si cae por un precipicio.

Tema 2: The Legend of Zelda 2D

- **Vista Top-Down:** Escena vista desde arriba. Ajustar la escala de gravedad (`Gravity Scale`) del `Rigidbody2D` del personaje a `0`.
- **Movimiento normalizado:** Control en 8 direcciones utilizando `Vector2` normalizado en `FixedUpdate()` para asegurar que la velocidad diagonal sea idéntica a la cardinal.
- **Ataque de espada:** Al presionar la tecla de ataque, activar temporalmente un objeto hijo que posea un `Collider2D` de tipo Trigger que represente el alcance de la espada. Al entrar en contacto con enemigos, debe invocar la función de resta de vida.
- **Cofres interactivos:** Crear cofres con `Is Trigger`. Al estar cerca y presionar la tecla de interacción, el cofre entrega un objeto sumándolo a la clase `Inventario` C#.

Tema 3: Pokémon 2D

- **Movimiento paso a paso (Grid/Paso):** Configurar el movimiento del personaje en vista aérea con `Gravity Scale = 0`, desplazándose por bloques discretos o pasos continuos hacia una dirección.
- **Zona de pasto alto:** Definir áreas de pastizal mediante `Collider2D` configurados como `Is Trigger`.
- **Cálculo de encuentro:** Mientras el personaje se desplace dentro del pasto alto, cada determinado paso o intervalo debe llamar a un método de la clase pura en C# que calcule la probabilidad matemática de iniciar un combate salvaje e imprima la alerta por consola.

Tema 4: Street Fighter 2D

- **Plano de pelea 1D horizontal:** Configurar al luchador limitando su desplazamiento al eje X dentro de una vista lateral. Congelar su rotación en Z.
- **Proyectil (Hadoken):** Instanciar un prefab del proyectil mediante `Instantiate()`. El proyectil debe contar con `Rigidbody2D` Kinematic o Dynamic sin gravedad y mover hacia adelante.
- **Hurtboxes y Hitboxes:** La bola de energía debe tener un `Collider2D` en modo `Is Trigger`. Al solaparse con el colisionador del oponente, descontar salud en la clase C# del luchador rival y destruir el proyectil.

Tema 5: Doom 2D

- **Desplazamiento y apuntado:** Movimiento fluido en la escena con `Rigidbody2D` (`Gravity Scale = 0`).
- **Dirección de disparo:** Calcular la dirección desde el personaje hacia la posición del cursor en pantalla haciendo uso de `Camera.main.ScreenToWorldPoint(Input.mousePosition)`.
- **Disparo y recolectables:** Instanciar balas físicas que salgan impulsadas hacia esa dirección. Disponer en el mapa sensores (`Is Trigger`) de paquetes de munición y botiquines que consuman el objeto y actualicen las variables del modelo C#.

Tema 6: Minecraft 2D

- **Plataformero con física:** Movimiento lateral y salto para un personaje con `Rigidbody2D` Dynamic.
- **Minería con Raycast:** Al hacer clic izquierdo con el mouse sobre un bloque del escenario, lanzar un `Physics2D.Raycast` desde la posición del personaje hacia el cursor para validar la distancia.
- **Desgaste de herramienta y Drops:** Al romper un bloque, reducir la durabilidad de la herramienta en la clase C# correspondiente e instanciar un prefab "Drop" en el mundo que caiga con física para ser recolectado.

Tema 7: Dark Souls 2D

- **Gestión de Estamina:** Toda acción física (correr, saltar, rodar) debe consultar primero a la clase pura C# si existe estamina disponible.
- **Rodaje e Invulnerabilidad:** Al presionar el botón de esquivar, aplicar un impulso físico horizontal de aceleración e invocar un estado temporal de invulnerabilidad donde se desactiven las colisiones con los enemigos.
- **Hoguera / Checkpoint:** Colocar un elemento interactivo con `Is Trigger` que, al ser activado por el jugador, reestablezca por completo la salud y la estamina en las clases de C#.

Tema 8: Resident Evil 2D

- **Control pesados o Top-Down pesados:** Configurar al jugador con `Gravity Scale = 0`. El movimiento debe ser lento y pausado simulando tensión.
- **Enemigos zombi:** Implementar zombis con `Rigidbody2D` o detectores que sigan al personaje. Si mantienen contacto continuo (`OnCollisionStay2D`), deben llamar periódicamente al método de recibir daño del personaje.

- **Llaves e Inventario:** Distribuir elementos por el escenario con `Is Trigger` que permitan recoger llaves. El personaje solo podrá atravesar puertas con Trigger si posee la llave correspondiente en la clase C# `Inventario`.

Tema 9: Hollow Knight 2D

- **Plataformas de precisión:** Control lateral ágil y salto con respuesta inmediata en la física.
- **Mecánica de Pogo Jump:** Al atacar hacia abajo en el aire y colisionar la espada (`Is Trigger`) contra un enemigo o un pincho, aplicar de inmediato una fuerza vertical hacia arriba al jugador mediante `AddForce()`.
- **Recarga de Alma:** El contacto exitoso del ataque contra un enemigo debe notificar a la clase C# para recargar la barra de "Alma" o energía mágica.

Tema 10: Pac-Man 2D

- **Laberinto sin gravedad:** Vista superior con `Gravity Scale = 0` sobre un laberinto con paredes estáticas (`Static Colliders`).
- **Validación de intersecciones:** Para girar en una esquina, utilizar pequeñas lecturas de `Physics2D.Raycast` en la dirección deseada para verificar que no haya una pared bloqueando el paso antes de cambiar el vector de movimiento.
- **Pellets y Fantasmas:** Consumir los puntos distribuidos en el laberinto detectándolos mediante `OnTriggerEnter2D` e incrementando la puntuación. Si un fantasma toca a Pac-Man, invocar la pérdida de vida en C#.

¡Éxitos a todos los equipos en esta segunda fase de desarrollo! Si surgen dudas técnicas, utilicen los canales oficiales del Discord de la cátedra.